SMARTQUEUE ML / CINETRUX

Estado del desarrollo, arquitectura y ruta de Machine Learning

Documento de transferencia técnica para revisión por IA y planificación de la siguiente fase

Propósito del documento
Consolidar lo que existe realmente en el repositorio, los contratos y endpoints disponibles, el nivel de avance verificable, las brechas actuales y una propuesta concreta para desarrollar el componente principal de Machine Learning sin confundir prototipo con producción.

Corte	Repositorio	Rama / commit	Validación
12 octubre 2026	DroxSebastianMR/smartqueue-frontend	main / aefa610	Frontend build OK; backend 27/27 tests

Lectura rápida

- La solución está organizada como monorepo con frontend React, backend NestJS y servicio ML Flask.
- La experiencia Cinetrux y el catálogo de cine están adelantados; Home consume datos del backend.
- El dominio SmartQueue conserva cola FIFO, reserva simulada, métricas, admisión y esquema experimental.
- El ML aún no está cerrado: existe contrato y script de entrenamiento, pero falta dataset reproducible, modelo entrenado, orquestación de ventanas e integración efectiva con admisión.
- Los pagos, disponibilidad real de butacas y comprobantes siguen bloqueados de forma intencional.

01 / Resumen ejecutivo y estado

SmartQueue ML es un prototipo académico de venta cinematográfica con cola virtual FIFO y control de admisión. Cinetrux es la capa de experiencia de usuario; el aporte investigable es comparar un límite fijo, un controlador reactivo y un controlador predictivo apoyado por Machine Learning para anticipar saturación a 30 segundos.

Estado honesto
La arquitectura base y buena parte del producto de cine ya existen. El componente ML está diseñado y parcialmente codificado, pero todavía no hay evidencia experimental ni un modelo operativo integrado al ciclo de admisión. El siguiente trabajo debe concentrarse en telemetría, simulación, etiquetado, entrenamiento, evaluación e integración.

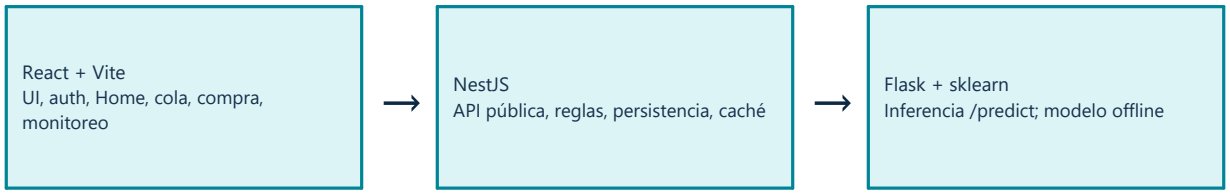
Inventario cuantitativo

Elemento	Cantidad / estado	Lectura
Aplicaciones	3	React/Vite, NestJS y Flask/scikit-learn
Endpoints NestJS	74	Públicos, autenticados y administrativos
Modelos Prisma	30	Cine, identidad, cola, venta y experimento
Enums Prisma	24	Estados y contratos del dominio
Variables ML	8	Ventanas operativas con horizonte de 30 s
Pruebas backend	27/27	11 suites superadas el 12/10/2026
Build frontend	Correcto	Advertencia no bloqueante: bundle principal > 500 kB
Pruebas ML	No ejecutadas	El runtime disponible no tenía pytest instalado

Semáforo de avance

Área	Estado	Observación
Arquitectura base	REALIZADA	Separación de procesos, módulos, contratos y ejecución conjunta.
Home Cinetrux	REALIZADA	Banner, catálogo, próximos estrenos, anuncios, promociones, sedes y horarios desde API.
Autenticación/RBAC	PARCIAL	JWT, sesiones y permisos; repositorio de identidad todavía en memoria.
Catálogo y funciones	PARCIAL	Lectura Prisma y administración; varias escrituras operativas aún son adaptadores en memoria.
Cola y compra simulada	BASE FUNCIONAL	FIFO, admisión, reserva y confirmación sin cobro.
Pagos/comprobantes	CONTRATO	Métodos listados; operaciones reales responden 503 deliberadamente.
Machine Learning	BASE TÉCNICA	Contrato, pipeline y API; falta cerrar el ciclo de datos y control.
Panel experimental	PARCIAL	Algunos KPIs vienen de API; riesgo, gráfica y comparación A/B/C siguen fijos en UI.

02 / Arquitectura de solución



Componente	Tecnología	Responsabilidad actual
smartqueue-frontend	React 19, Vite 7, TypeScript, Tailwind, React Query, Zustand, Axios	Home Cinetrux, autenticación, cola, compra simulada y panel experimental.
smartqueue-backend	NestJS 11, Prisma, PostgreSQL/Supabase, Redis opcional, Swagger	API, reglas de dominio, seguridad, persistencia, caché, cola, venta, métricas y cliente ML.
smartqueue-ai	Python 3.12, Flask, pandas, NumPy, scikit-learn, joblib	Entrenamiento offline y servicio de inferencia de saturación.

Principios y límites

- React consume una única API pública de NestJS; el navegador no llama directamente al servicio ML.
- NestJS es monolito modular para negocio; Flask se separa por lenguaje, dependencias y ciclo de entrenamiento.
- PostgreSQL es fuente de verdad; Redis/memoria se utiliza solo como caché de corta duración.
- La predicción ajusta el ritmo de ingreso, nunca el orden FIFO ni el inventario.
- Ante timeout, error o respuesta inválida del ML, el diseño exige volver al control reactivo.
- El modelo se entrena offline y Flask carga un Pipeline joblib; no se entrena dentro de /predict.

Estructura del repositorio

```
SmartQueue ML/ smartqueue-frontend/ # aplicación web modular smartqueue-backend/ # API NestJS y Prisma smartqueue-ai/ # entrenamiento e inferencia ML docs/ # arquitectura y catálogo de API scripts/dev.mjs # inicia backend y frontend con npm run dev docker-compose.yml # servicios auxiliares de desarrollo
```

Ejecución local

Desde la raíz, npm run dev inicia NestJS y Vite. URLs esperadas: frontend en 5173, API en 3000/api/v1, Swagger en 3000/api/docs y Flask en 5001 cuando se inicia por separado.

03 / Frontend: arquitectura y avance funcional

La web sigue una organización por feature y layouts, evitando la convención screens de React Native. Las rutas públicas, protegidas y administrativas están separadas; AuthLayout usa Outlet para login, registro y recuperación.

Capas principales

Área	Carpetas	Responsabilidad
Aplicación	app/providers, app/router	Providers, sesión, guards, rutas y constantes.
Layouts	landing, auth, booking, admin	Composición responsive por contexto.
Features	auth, home, queue, purchase, monitoring	Pages, componentes, hooks, tipos, enums, formularios y servicios.
Infraestructura	infrastructure/api	Axios, interceptores, endpoints, servicios HTTP, tipos API y mappers.
UI compartida	components/ui	Button, Card, Badge, Select, Carousel, Skeleton y Progress.

Home conectado al backend

- Banner dinámico: consume hasta 10 banners, resuelve su película y genera indicadores según la cantidad disponible.
- Cartelera y próximos estrenos: carruseles reutilizables con datos de /movies/now-playing y /movies/upcoming.
- Buscador: película, ciudad, sede y fecha; consulta /showtimes y conserva selección con Zustand.
- Anuncios y promociones: datos remotos; la promoción utiliza composición visual de ancho completo.
- React Query maneja carga/caché de servidor; no se muestran películas locales inventadas si la API falla.
- Axios agrega Bearer, normaliza errores y limpia sesión ante 401. Los enums de frontend replican valores del backend.

Rutas visibles

Ruta	Acceso	Estado
/	Público	Home Cinetrux conectado al catálogo
/login, /register, /recover	Público	Pantallas separadas bajo AuthLayout
/queue	Sesión	Sala de espera / estado de cola
/purchase	Sesión	Compra y selector de butacas de prototipo
/monitoring	ADMIN	Panel experimental; aún contiene valores fijos

Deuda visible
MonitoringPage todavía contiene barras, riesgo 72 %, versión de modelo y tabla comparativa codificados en el frontend. Deben reemplazarse por ventanas, predicciones y resultados A/B/C persistidos antes de presentar evidencia.

04 / Backend: módulos, seguridad y persistencia

NestJS usa módulos con carpetas directas controllers, services, repositories, dto, enums, guards, decorators, types, constants y utils. Incluye ValidationPipe estricto, Helmet, CORS configurado, prefijo /api/v1, filtro global de excepciones y Swagger.

Módulos registrados

Grupo	Módulos	Alcance
Identidad	auth	Registro, login, JWT, refresh, sesiones, recuperación y permisos.
Portada/catálogo	banners, movies, advertisements, promotions, catalog	Contenido editorial, películas, versiones y assets.
Operación de cine	cinemas, showtimes	Sedes, salas, butacas, funciones y horarios.
Transacción	events, queue, sales, admission	Evento, turnos FIFO, reservas, compra simulada y admisión.
Observabilidad/ML	metrics, ml-client, health	Dashboard, contrato HTTP con Flask y salud.
Pagos	payments	Catálogo de métodos y contratos bloqueados de pago/comprobante.

Autenticación y autorización

Rol	Alcance previsto
CUSTOMER	Perfil y operaciones propias
BOX_OFFICE	Venta/devolución en taquilla
USHER	Validación de entradas y consulta operativa
CINEMA_MANAGER	Operación de una sede; el alcance por cinemald aún falta
OPERATIONS_ANALYST	Métricas y monitoreo en lectura
ADMIN	Administración global

Caché

CacheStoreModule soporta memoria acotada o Redis. Se cachean intentos de login (60 s), catálogo (60 s), dashboard (3 s) y estado de cola (2 s). Si Redis es solicitado y no está disponible, el backend falla al iniciar; no cambia silenciosamente a memoria. Redis no sustituye PostgreSQL.

Persistencia Prisma

El esquema contiene 30 modelos y 24 enums. Los dominios principales son:

Dominio	Modelos clave
Identidad	User, UserRoleAssignment, AuthSession, PasswordResetRequest
Cine	Cinema, Auditorium, Seat, Movie, MovieRelease, MovieLocalization, Genre, MovieAsset, MovieVersion, Showtime
Editorial	MovieCatalogPlacement, Banner, Advertisement, Promotion
Venta	Ticket, BookingOrder, PaymentAttempt, FiscalReceipt
SmartQueue	Event, QueueEntry, PurchaseReservation, Purchase
Experimento	ExperimentRun, MetricWindow, PredictionLog

Limitación de producción
La identidad sigue en memoria y algunas operaciones administrativas de cine también. La disponibilidad real de butacas, cobro, webhooks, emisión SUNAT/OSE, correo y autorización por sede no están implementados.

05 / Inventario de endpoints - públicos y autenticación

Método	Ruta	Uso / estado
GET	/health	Salud de NestJS
GET	/banners	Banners activos, paginados
GET	/advertisements	Anuncios activos
GET	/promotions	Promociones activas
GET	/movies	Búsqueda por título/género
GET	/movies/now-playing	Películas en cartelera
GET	/movies/upcoming	Próximos estrenos
GET	/movies/rankings/top-selling	Devuelve NO_SALES_DATA hasta integrar ventas
GET	/movies/:slug	Ficha de película
GET	/movies/:slug/trailers	Avances / assets de video
GET	/cinemas	Sedes con filtros
GET	/cinemas/nearby	Cercanía por coordenadas; distancia en línea recta
GET	/cinemas/:slug	Detalle de sede
GET	/cinemas/:slug/auditoriums	Salas de una sede
GET	/auditoriums/:id/seats	Plano físico de butacas
GET	/showtimes	Funciones por película/sede/ciudad/fecha
GET	/showtimes/:id	Detalle de función
GET	/showtimes/:id/availability	NOT_CONNECTED hasta reservas atómicas
GET	/cinemas/:slug/showtimes	Funciones por cine
GET	/movies/:slug/showtimes	Funciones por película
GET	/movies/:slug/schedule	Agenda de 1 a 5 días
GET	/payments/methods	Métodos previstos; no configurados
GET	/receipts/types	Boleta/factura previstas
GET	/events/:slug	Evento transaccional de demostración

Autenticación

Método	Ruta	Resultado
POST	/auth/register	Crea cliente y sesión temporal
POST	/auth/login	Access + refresh token
POST	/auth/refresh	Rotación de sesión
GET	/auth/me	Usuario, roles y permisos
GET	/auth/permissions	Matriz efectiva
POST	/auth/logout	Revoca sesión actual
POST	/auth/logout-all	Revoca todas las sesiones
GET	/auth/sessions	Sesiones activas
DELETE	/auth/sessions/:id	Revoca sesión propia
POST	/auth/password/change	Cambia clave y revoca sesiones
POST	/auth/password/forgot	Token de desarrollo; falta email
POST	/auth/password/reset	Consume token de recuperación

Contrato transversal

- Access token HS256 de 1 hora y refresh opaco de 7 días, rotado al renovar.
- ValidationPipe transforma tipos, elimina propiedades no permitidas y rechaza campos extra.
- Formato global de error: statusCode, error, code, message, path, timestamp y meta opcional.
- Códigos de validación y AUTH_* están tipados y documentados en Swagger.
- La persistencia de usuarios/sesiones todavía debe migrarse del repositorio en memoria a Prisma.

06 / Inventario de endpoints - operación y administración

Todas las rutas NestJS llevan el prefijo /api/v1. Las listas principales aceptan page/limit con máximo 10 y responden data + meta. Las rutas /admin requieren Bearer JWT ADMIN. Swagger está disponible en /api/docs y el contrato OpenAPI en /api/docs-json.

Método	Ruta	Uso / estado
POST	/queue/entries	Ingreso idempotente a cola FIFO
GET	/queue/events/:eventId/status	Posición y estado por token
POST	/admission/events/:eventId/run	Admite N cupos; hoy N llega en la solicitud
POST	/sales/reservations	Reserva temporal simulada
POST	/sales/reservations/:id/confirm	Confirmación idempotente sin cobro
GET	/metrics/dashboard	Cola, compradores, compras y última predicción
POST	/payments/intents	JWT; 503 hasta proveedor/orden
POST	/receipts/requests	JWT; 503 hasta emisión tributaria

Administración de películas

Método	Ruta
GET / POST	/admin/movies
GET / PATCH / DELETE	/admin/movies/:slug
POST	/admin/movies/:slug/publish
POST	/admin/movies/:slug/unpublish
PUT / DELETE	/admin/movies/:slug/placements/:section
GET / POST	/admin/movies/:slug/versions
DELETE	/admin/movies/:slug/versions/:id
GET / POST	/admin/movies/:slug/assets
DELETE	/admin/movies/:slug/assets/:id

Administración de sedes, salas y funciones

Método	Ruta
GET / POST	/admin/cinemas
PATCH / DELETE	/admin/cinemas/:slug
GET / POST	/admin/cinemas/:slug/auditoriums
PATCH / DELETE	/admin/auditoriums/:id
POST	/admin/auditoriums/:id/seats
DELETE	/admin/auditoriums/:id/seats/:seatId
GET / POST	/admin/showtimes
PATCH	/admin/showtimes/:id
POST	/admin/showtimes/:id/on-sale
POST	/admin/showtimes/:id/cancel

07 / Machine Learning: lo que ya existe

El objetivo definido es clasificación binaria anticipada: estimar si habrá saturación durante los siguientes 30 segundos usando información disponible al cierre de una ventana operativa.

Contrato de entrada

Variable	Definición
arrival_rate	Solicitudes nuevas por segundo
arrival_trend	Cambio respecto de la ventana anterior
queue_length	Participantes esperando
active_buyers	Permisos de compra activos
admission_limit	Límite vigente de compradores
latency_p95_ms	Percentil 95 de latencia de compra
error_rate	Solicitudes de compra fallidas / totales
completion_rate	Compras completadas por segundo

API Flask

Ruta	Contrato
GET /health	status, service y model_loaded
POST /predict	schema_version=1.0, window_end y ocho variables; devuelve risk_score, HEALTHY/SATURATION, horizon_seconds=30 y model_version

Entrenamiento implementado

- Lee CSV y exige las ocho variables, run_id y saturation_label.
- Divide por ejecución mediante GroupShuffleSplit para evitar mezclar ventanas de una misma corrida entre train y test.
- Pipeline: imputación por mediana + StandardScaler + modelo.
- Compara Regresión Logística balanceada y KNN ponderado por distancia.
- Selecciona por F1, reporta recall/F1 y guarda saturation_pipeline.joblib + model_metadata.json.
- Si el modelo no está disponible, /predict devuelve 503 MODEL_UNAVAILABLE; no inventa una heurística.

Piezas preparadas en backend

- HttpMlClientService con timeout configurable y validación de riesgo 0..1/horizonte 30 s.
- MlClientModule importado por AdmissionModule.
- nextAdmissionLimit implementa límites FIXED, REACTIVE y PREDICTIVE con umbrales acotados.
- Prisma incluye ExperimentRun, MetricWindow y PredictionLog para trazabilidad.
- Dashboard puede recuperar la predicción más reciente si existe en PostgreSQL.

Brecha crítica
RunAdmissionService no usa HttpMlClientService ni nextAdmissionLimit. Solo recibe slots y admite esa cantidad en FIFO. Tampoco existe todavía un productor de MetricWindow ni una orquestación que ejecute una predicción por ventana y persista decisión/fallback.

08 / Brechas verificadas antes de entrenar

Prioridad	Brecha	Consecuencia
P0	No hay generador de demanda/carga versionado	No existe dataset reproducible ni escenarios comparables.
P0	No se calculan y persisten MetricWindow	El modelo no tiene observaciones reales/sintéticas trazables.
P0	No está implementada la etiqueta futura	No puede entrenarse sin leakage ni circularidad.
P0	Admisión no invoca al ML	El modo predictivo no opera aunque Flask responda.
P0	No hay artefacto entrenado versionado	Flask devuelve MODEL_UNAVAILABLE.
P1	Panel contiene resultados fijos	La demostración puede confundir diseño con evidencia.
P1	Evaluación solo calcula recall y F1	Faltan precisión, PR-AUC, matriz, calibración y umbral.
P1	Sin registro de timeout/fallback/antigüedad	No se puede auditar confiabilidad operacional.
P1	Sin protocolo A/B/C automatizado	No hay comparación justa entre fijo, reactivo y predictivo.
P2	Sin monitoreo de drift/reentrenamiento	Aceptable para MVP, pero debe declararse como límite.

Definición de etiqueta recomendada

Para una ventana que termina en t, usar solamente variables observadas hasta t. Definir y=1 si alguna de las tres ventanas siguientes (t+10, t+20, t+30 s) supera el umbral de latencia L o la tasa de errores E; y=0 en caso contrario. Congelar L y E después de un piloto. Excluir ventanas sin horizonte futuro completo o sin volumen suficiente para métricas estables.

Regla metodológica
La predicción nunca debe usarse para fabricar la etiqueta. Las ventanas de una misma ejecución no pueden repartirse entre entrenamiento y prueba. El conjunto de prueba debe permanecer cerrado hasta congelar modelos, hiperparámetros y umbral.

Dataset mínimo versionado

Grupo	Columnas recomendadas
Identidad	run_id, window_end, scenario, seed, generator_version, control_mode
Configuración	stock, duration_s, min_limit, max_limit, current_limit, capacity_profile
Features	Las ocho variables del contrato v1
Etiqueta	saturation_label, label_latency_threshold_ms, label_error_threshold
Auditoría	evaluable, exclusion_reason, schema_version

09 / Ruta recomendada para desarrollar el ML

Fase	Trabajo	Entregable verificable
1. Congelar contrato	Acordar ventana=10 s, horizonte=30 s, L/E, mínimos de muestra y unidades.	Especificación v1 + validadores compartidos.
2. Instrumentar	Middleware/interceptor de métricas de compra; agregador por ventana; persistencia MetricWindow.	Ventanas coherentes consultables en PostgreSQL.
3. Simular	Generador lógico y carga HTTP con perfiles normal, ráfaga y pico; semillas y configuración.	Al menos 30 ejecuciones iniciales, repetibles.
4. Etiquetar/exportar	Job que calcula futuro, marca no evaluables y exporta CSV/Parquet versionado.	Dataset + diccionario + informe de calidad.
5. Entrenar	Baseline trivial, Logística y KNN; validación agrupada; búsqueda de umbral.	Pipeline y metadata reproducibles.
6. Evaluar	Recall, precisión, F1, PR-AUC, matriz, calibración, latencia de inferencia.	Informe sin tocar el test final.
7. Integrar	Orquestador por ventana llama /predict, registra respuesta y aplica policy con fallback.	PREDICTIVE operativo y auditable.
8. Experimentar	Mismas cargas/semillas para FIXED, REACTIVE y PREDICTIVE; repeticiones emparejadas.	Tabla y gráficos calculados, no hardcodeados.
9. Presentar	Conectar MonitoringPage a resultados y documentar limitaciones.	Demo y paper reproducibles.

Orden de implementación inmediata

La siguiente iteración no debería comenzar ajustando KNN o una red neuronal. Debe comenzar construyendo datos confiables. El modelo más simple que supere al baseline y al controlador reactivo bajo el protocolo definido será más defendible que un modelo complejo sin trazabilidad.

Arquitectura objetivo del ciclo

```
Generador de carga -> endpoints de cola/compra -> telemetría cruda -> agregador cada 10 s -> MetricWindow -> etiquetador offline (mira 30 s al futuro) -> dataset versionado -> entrenamiento y evaluación -> artifact + metadata -> Flask /predict -> orquestador NestJS -> nextAdmissionLimit + fallback -> PredictionLog + dashboard
```

10 / Diseño experimental propuesto

Pregunta central: ¿en qué medida un controlador predictivo reduce el deterioro operacional frente a una cola fija y un controlador reactivo, bajo las mismas condiciones de carga y recursos?

Escenarios

Escenario	Perfil	Objetivo
Normal	Llegadas estables por debajo de capacidad	Medir falsas alarmas y restricción innecesaria.
Ráfaga	Pico corto y recuperación	Medir reacción temprana y tiempo de recuperación.
Pico sostenido	Demanda por encima de capacidad	Medir errores, latencia y throughput bajo estrés.
Escalonado	Incrementos y descensos controlados	Medir histéresis y estabilidad del límite.

Comparación justa

- Usar las mismas semillas, trazas de llegada, stock, duración, recursos y configuración por cada modo.
- Repetir cada combinación al menos cinco veces; reportar media, mediana, dispersión e intervalos.
- Separar carga de espera, compra y consulta para que el polling no distorsione el endpoint crítico.
- Preservar FIFO y verificar que no existan turnos o compras duplicadas.
- Registrar fallos del ML, timeout, fallback, versión y antigüedad de la predicción.

Métricas

Dimensión	Métricas
Predictiva	Recall, precisión, F1, PR-AUC, matriz de confusión, Brier/calibración
Rendimiento	Latencia p50/p95/p99, error rate, throughput, completadas
Experiencia	Espera media/p95, abandonos, tiempo hasta compra
Control	Cambios del límite, tiempo en mínimo/máximo, oscilación, fallback
Equidad	Violaciones FIFO, duplicados, expiraciones y diferencias por cohorte

Metas iniciales para validar, no resultados

- Recall de saturación >= 0.80 y F1 >= 0.75 en test, si el dataset contiene suficientes positivos.
- Reducir al menos 15 % la tasa de error frente a FIXED en escenarios exigentes sin disminuir más de 5 % las compras completadas.
- Superar o complementar al REACTIVE; si no hay mejora, documentar resultado negativo y causas.
- Inferencia dentro del timeout de 1 s, con fallback probado y sin romper FIFO.

Criterio de éxito científico
El éxito no es demostrar que ML siempre gana. Es producir una comparación reproducible, sin fuga de datos, donde pueda aceptarse o rechazarse la hipótesis con evidencia.

11 / Riesgos, decisiones y deuda técnica

Riesgo	Mitigación
Fuga temporal	Features solo hasta t; etiqueta en t+30; partición por run_id.
Sesgo de escenarios	Variar perfiles, capacidad, seeds y conservar escenarios inéditos.
Desbalance	Reportar soporte por clase, PR-AUC y usar class_weight/umbrales.
Modelo no calibrado	Curva de calibración/Brier; calibrar antes de interpretar risk_score como probabilidad.
Oscilación del controlador	Histéresis, pasos acotados, mínimos/máximos y cooldown.
Dependencia del ML	Timeout, circuit breaker/fallback reactivo y estado visible.
Panel engañoso	Eliminar constantes y mostrar solo datos etiquetados como medidos/sintéticos.
Credenciales	Mantener secretos en .env; rotar cualquier credencial expuesta.
Medios sin licencia	Usar fuentes autorizadas y documentar origen de imágenes/tráilers.
Reserva concurrente	Transacción, hold con expiración y restricción de unicidad activa por función/butaca.

Decisiones que deben cerrarse antes del dataset definitivo

- Umbral de latencia L y tasa de errores E para saturation_label.
- Volumen mínimo por ventana para considerarla evaluable.
- Duración exacta de ejecuciones y número de repeticiones por combinación.
- Capacidad base, min/max de admisión, TTL de acceso/reserva y patrón de servicio.
- Qué representa una compra fallida: timeout, 5xx, rechazo por stock o validación.
- Criterio primario de selección: F1, recall sujeto a precisión mínima o costo operacional.
- Estrategia de despliegue del artefacto y compatibilidad schema/model_version.

Trabajos paralelos que no deben bloquear el ML

Persistir identidad, completar CRUD Prisma, pagos, comprobantes y diseño comercial son líneas valiosas, pero el experimento puede avanzar con compra simulada siempre que el camino crítico genere telemetría real y reproducible.

12 / Bloque listo para entregar a otra IA

Puede copiarse el siguiente encargo junto con este PDF:

```
Actúa como arquitecto de Machine Learning y MLOps para SmartQueue ML. Usa este documento como fuente de verdad del estado
actual; no asumas que las capacidades marcadas como pendientes ya existen.

Objetivo: proponer y luego implementar, por etapas verificables, el ciclo completo de datos e inferencia para predecir
saturación a 30 segundos y comparar FIXED, REACTIVE y PREDICTIVE sin alterar FIFO.

Primero entrega:
1. Auditoría del contrato actual de ocho features y propuesta de unidades/rangos.
2. Diseño del recolector/agregador de ventanas de 10 segundos en NestJS.
3. Definición exacta de saturation_label sin leakage y reglas de exclusión.
4. Diseño del generador de demanda y protocolo reproducible de escenarios/seeds.
5. Esquema del dataset versionado y consultas/exportación desde Prisma.
6. Plan de entrenamiento: baseline trivial, Logística y KNN; partición por run_id, validación, umbral y métricas.
7. Integración de Flask con el orquestador de admisión, persistencia de PredictionLog, timeout y fallback reactivo.
8. Plan de pruebas unitarias, integración, carga y experimento A/B/C.
9. Cambios para que MonitoringPage muestre únicamente resultados calculados.

Restricciones:
- No usar la predicción para crear la etiqueta.
- No mezclar ventanas de una corrida entre train y test.
- NestJS decide; Flask solo predice.
- La predicción cambia el ritmo, no el orden FIFO.
- Sin un modelo válido debe usarse control reactivo y registrarse el fallback.
- No presentar datos sintéticos como resultados comerciales reales.
- Mantener el contrato schema_version/model_version y la arquitectura existente.

Antes de editar código, identifica qué archivos se reutilizan, qué módulos faltan y propone una secuencia de commits pequeños.
Señala cualquier contradicción entre el código y este informe.
```

Resultado esperado de la siguiente fase
Un dataset reproducible, un Pipeline evaluado, una integración predictiva con fallback y un panel que compare modos usando registros reales del experimento. Solo después conviene optimizar modelos o ampliar variables.

Fuentes internas revisadas

README raíz y de cada aplicación; docs/ARCHITECTURE.md; docs/CINEMA_API.md; controladores NestJS; schema y migración Prisma; servicios de admisión, métricas y ml-client; código de smartqueue-ai; rutas, infraestructura API y panel de React; documento de concepto SmartQueue ML v1.0.

Este informe describe el commit aefa610 de main y las verificaciones ejecutadas el 12 de octubre de 2026.